

Disadvantages of Pointers :-

- Uninitialized Pointers
- The pointer may cause a memory leak
- Dangling Pointers

Uninitialized pointers in C++ :-

```
int *p;
```

if we have declared a pointer then we should not use that pointer unless we have initialized it.

```
*p = 25;
```

This means that we want to store the value '25' wherever the pointer is pointing. But the question here is where the pointer is pointing? In, 'int *p' it is not pointing anywhere. Just we have declared. Then what is the address in 'p' here?

Some default garbage addresses may be in 'p', some random addresses that may belong to a program or may not belong to a problem. So, it is an invalid address as we have not made a pointer to point to some particular location. So first of all, make it point to some location then we can access it. There are 3 methods of doing it.

1st Method :-

```
int x = 10;          int *p = &x;
```

If we have some variable 'x' then, Now the pointer is pointing to this known variable 'x' which is already declared.

2nd Method :-

```
int *p = (int*) 0x5628;
```

We can assign some addresses using some hexadecimal codes but that address has to be type casted as an integer pointer. So, can we directly assign some addresses to a pointer? Yes, if we are sure that the address belongs to the program so this type of initialization is also allowed. **This is not commonly used. This is mostly used in systems programming**

3rd Method :-

```
int *p = new int[5];
```

We can dynamically allocate some memory and assign that to a pointer. If we don't write size and write only 'int' then it will allocate just one integer so either to an existing variable.

Memory Leak :-

This is related to a pointer as well as heap memory. As we have already discussed heap memory, when we are allocating heap memory then when we don't require that, we should deallocate it. If we don't deallocate it then we say that memory is leaked from that total set of memory.

```
int *p = new int[3];           p = NULL;
```

Here we have a pointer and I have allocated heap memory of some size. Then after some time, we don't need this memory. So, we simply say 'p = NULL', then point P will not be pointing on that memory. We should not do this unless we have explicitly deleted the memory. So first of all, say delete 'p' then only, make 'p' as null.

```
delete []p;
```

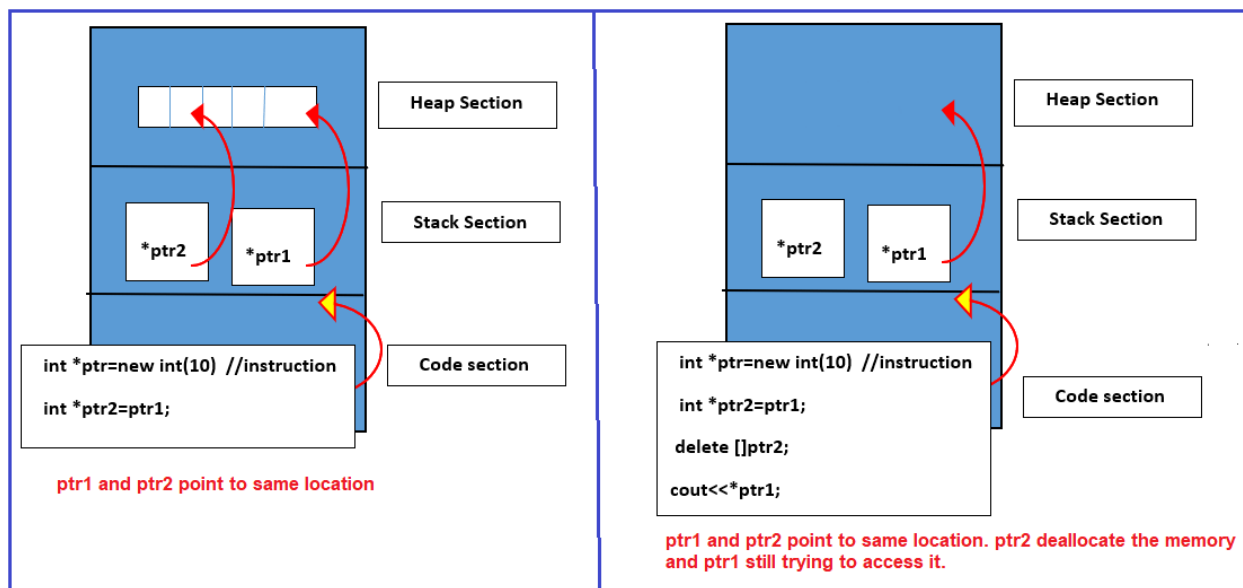
```
p = NULL;
```

Now here is one more thing that we can write 'p = 0' also or write 'p = nullptr'.

In modern C++ it is suggested to use 'nullptr'. You should avoid using null. So back to this memory leak, the conclusion is you must delete the memory when you are not using it before making a pointer to null. Now let us move to the third problem which is the dangling pointer.

Dangling Pointer in C++ :-

So uninitialized pointers mean, the pointer is never initialized, dangling pointer means the pointer was initialized but memory is deallocated. We should avoid these three types of problems while writing programs or developing applications. Actually, these problems are caused due to negligence of beginner programmers. Expert programmers may check all these things thoroughly only before delivering a program or before developing software.



```
#include<iostream>
using namespace std;
int main() {
    int *ptr1=new int(10);
    int *ptr2=ptr1;
    delete []ptr2;
    cout<<*ptr1;
    return 0;
}
```

The improper use of pointers is often at the root of many security problems. When an application behaves in unpredictable ways, it may not seem to be a security issue, at least in terms of unauthorized access.

Pointers can do arithmetic, References can't: **Memory access via pointer arithmetic is fundamentally unsafe and for safeguarding,** Java has a robust security model and disallows pointer arithmetic for this reason.

What are Smart Pointers in C++?

If you are creating anything inside the heap memory then for accessing the heap memory, we need to use pointers. The problem with heap memory is that when you don't need it then you must deallocate the memory. And mostly the programmer shows laziness in writing the code for the deallocation of objects from heap memory which causes severe problems like memory leaks which will cause the program to crash.



```
1  #include<iostream>
2  using namespace std;
3  class Rectangle{
4  private:
5      int Length;
6      int Breadth;
7  public:
8      Rectangle(int l, int b){
9          Length = l;
10         Breadth = b;
11     }
12     int Area(){
13         return Length * Breadth;
14     }
15 };
16
17 int Fun(int l, int b){
18     Rectangle *p = new Rectangle(l, b);
19     int area = p->Area();
20     delete p;
21     return area;
22 }
23 int main(){
24     while(1){
25         int Result = Fun(10, 20);
26         cout << Result << endl;
27     }
28     return 0;
29 }
```

Using Smart Pointers in C++:

If we declare the smart pointer then they will automatically deallocate the object when the smart pointer is going out of the scope. Let us show you how we can declare smart pointers in C++.

```
int Fun(int l, int b)
{
    unique_ptr<Rectangle> p(new Rectangle(l, b));
    int area = p->Area();
    return area;
}
```

Example to Understand unique_ptr in C++ :-

If you are using unique_ptr, if an object is created and a pointer is pointing to that object then only one pointer can point to that object. So, we cannot share this object with another pointer. But we can transfer the control from one pointer to another pointer by removing p1. So unique_ptr means upon an object at a time only one pointer will be pointing.

```
1  #include<iostream>
2  #include<memory>
3  using namespace std;
4  class Rectangle{
5  private:
6      int Length;
7      int Breadth;
8  public:
9      Rectangle(int l, int b){
10         Length = l;
11         Breadth = b;
12     }
13     int Area(){
14         return Length * Breadth;
15     }
16 };
17
18 int main(){
19     unique_ptr<Rectangle> ptr1(new Rectangle(10,5));
20     cout<<ptr1->Area()<<endl;
21
22     unique_ptr<Rectangle> ptr2;
23     ptr2=move(ptr1);
24
25     cout<<ptr1->Area()<<endl;
26     cout<<ptr2->Area()<<endl;
27     return 0;
28 }
```

Output: 50

Shared_ptr :-

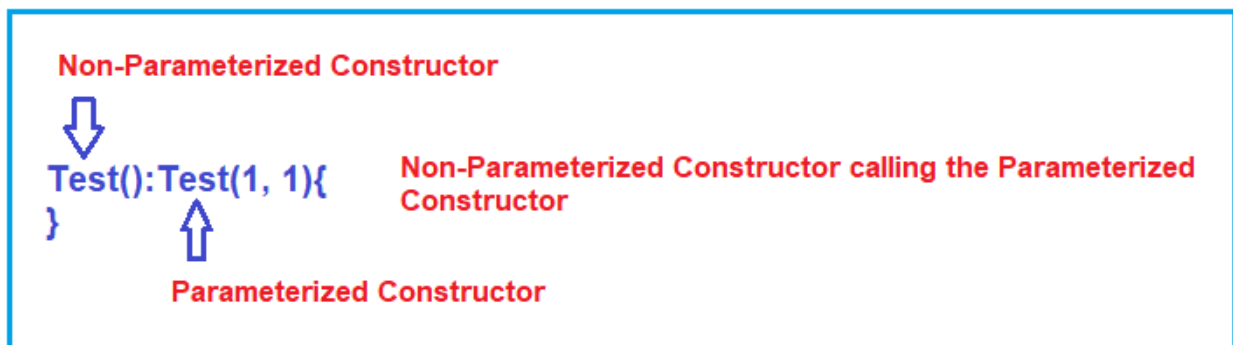
Just like how we have used `unique_ptr`, the same way we have to use `shared_ptr`. More than one pointer can point to one object. This pointer maintains a `Ref_count` that is a reference counter. Suppose 3 pointers are pointing on a single object the `Ref_count` will be 3. So `shared` means an object can be used by more than one pointer. If we remove one pointer then `Ref_count` will be reduced by 1. We can know the value of `Ref_count` by using the `use_count()` function.

```
1  #include<iostream>
2  #include<memory>
3  using namespace std;
4  class Rectangle{
5  private:
6      int Length;
7      int Breadth;
8  public:
9      Rectangle(int l, int b){
10         Length = l;
11         Breadth = b;
12     }
13     int Area(){
14         return Length * Breadth;
15     }
16 };
17
18 int main(){
19     shared_ptr <Rectangle> ptr1 (new Rectangle(10, 5));
20     cout << ptr1->Area() << endl;
21
22     shared_ptr <Rectangle> ptr2;
23     ptr2 = ptr1;
24
25     cout << "ptr1 " << ptr1->Area() << endl;
26     cout << "ptr1 " << ptr2->Area() << endl;
27     cout << ptr1.use_count() << endl;
28 }
29 //Output
30 50
31 ptr1 50
32 ptr1 50
33 2
```

Weak_ptr :-

It is also the same as shared_ptr. Here also more than one pointer can point to a single object. But it will not maintain Ref_count. So that's why it is known as weak_ptr. So, the pointer will not have a strong hold on the object. The reason is if the pointers are holding the object and requesting other objects, they may form a deadlock between the pointers. So, to avoid deadlock, weak_ptr is useful. So, it doesn't have Ref_count so it is more like unique_ptr but it allows the pointer to share an object, so it is more like shared_ptr. It is in between unique and shared which is not strict. It doesn't bother how many pointers are pointing at an object.

InClass Initializer and Delegation of Constructors :-





```

1  #include<iostream>
2  using namespace std;
3  class Test{
4  private:
5      int x = 15, y = 30, z;
6  public:
7      Test(int a, int b){
8          x = a;
9          y = b;
10     }
11     //using constructor delegation
12     Test():Test(35, 75){
13         z = 10;
14     }
15     void Display(){
16         cout<<"x : "<<x<<", y : "<<y<<", z : "<<z;
17     }
18 };
19 int main(){
20     Test obj;
21     obj.Display();
22 }

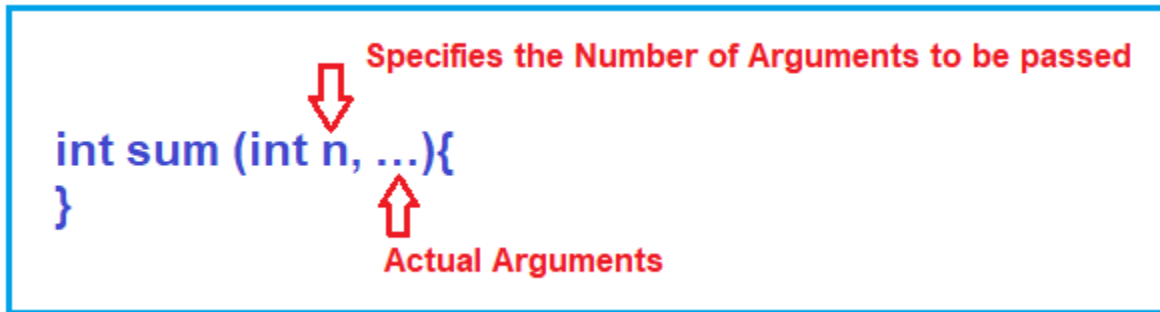
```

Ellipsis in C++:

Ellipsis is used for taking a variable number of arguments in a function. For example, if we want to write a function for finding the sum of elements or sum of integers then we want our function to work with a different number of parameters like,

Sum (10, 20) = 30

Sum (30, 54, 23) = 107



How to access Elements using Ellipsis in C++?

For accessing the elements, there is a class available in C++ that is `va_list`. For a better understanding, please have a look at the following code. What are the important instructions in this function?

`va_list()`, `va_start()`, `va_arg()` and `va_end()`.

```
int Sum(int n, ...){
    va_list list;
    va_start(list, n);
    int s = 0;

    for(int i = 0; i < n; i++)
        s += va_arg(list, int);

    va_end(list);
    return s;
}
```



```
1  #include<iostream>
2  #include<cstdarg>
3  using namespace std;
4  int sum (int n, ...){
5      va_list list;
6      va_start (list, n);
7      int x;
8      int s = 0;
9      for (int i = 0; i < n; i++){
10         x = va_arg (list, int);
11         s += x;
12     }
13     return s;
14 }
15 int main(){
16     cout << sum (3, 12, 24, 36) << endl;
17     cout << sum (7, 13, 26, 39, 52, 65, 78, 81) << endl;
18 }
19 //Output
20 72
21 354
```